MAJOR PROJECT REPORT

"Secure Student Portal with Authentication and Web Security Controls"

Submitted To: Ediglobe

Submitted By,

Name: **L V Nivethan**

Batch: **April Batch – Cyber Security**

Submission Date: **03/06/2026**

ABSTRACT

Modern web applications frequently handle sensitive user information such as login credentials and personal data. Poor implementation of authentication and security controls can expose applications to various cyber threats including SQL Injection, Cross-Site Scripting (XSS), session hijacking, and unauthorized access.

This project presents the development of a Secure Student Portal that provides user registration, login, profile management, and secure session handling. The application incorporates industry-standard security measures such as password hashing, input validation, JWT-based authentication, rate limiting, security headers, and protection against common OWASP Top 10 vulnerabilities.

The project was developed using React, Node.js, Express.js, and Supabase PostgreSQL, and deployed using Vercel.

TABLE OF CONTENTS

---

1. INTRODUCTION

Web applications are widely used across educational institutions, businesses, and online platforms. As the number of cyber threats continues to increase, it is important to ensure that web applications are developed with security in mind.

This project focuses on building a secure student portal that allows users to register, log in, and access protected resources while implementing essential cybersecurity practices. The project demonstrates how security can be integrated throughout the software development lifecycle rather than added as an afterthought.

---

2. PROBLEM STATEMENT

Many beginner web applications focus only on functionality and often neglect security considerations. This can result in vulnerabilities that allow attackers to gain unauthorized access, steal data, or compromise system integrity.

The objective of this project is to design and implement a secure authentication-based web application that protects against common web vulnerabilities while maintaining usability and performance.

---

3. OBJECTIVES

The main objectives of this project are:

- Develop a secure user registration system.
- Implement secure user authentication.
- Store passwords using strong hashing algorithms.
- Prevent SQL Injection attacks.

- Prevent Cross-Site Scripting (XSS) attacks.
- Implement secure session management using JWT.
- Restrict access to protected resources.
- Implement input validation and sanitization.
- Apply security best practices recommended by OWASP.
- Deploy the application securely on a cloud platform.

---

4. SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel i3 or higher
- RAM: 4 GB minimum
- Storage: 1 GB available space
- Internet Connection

Software Requirements

- Visual Studio Code
- Node.js
- React
- Express.js
- PostgreSQL
- Supabase
- GitHub
- Vercel
- Modern Web Browser

---

5. TECHNOLOGY STACK

Frontend

- React
- Bootstrap
- Axios
- React Router

Backend

- Node.js

- Express.js

Database

- Supabase PostgreSQL

Security Libraries

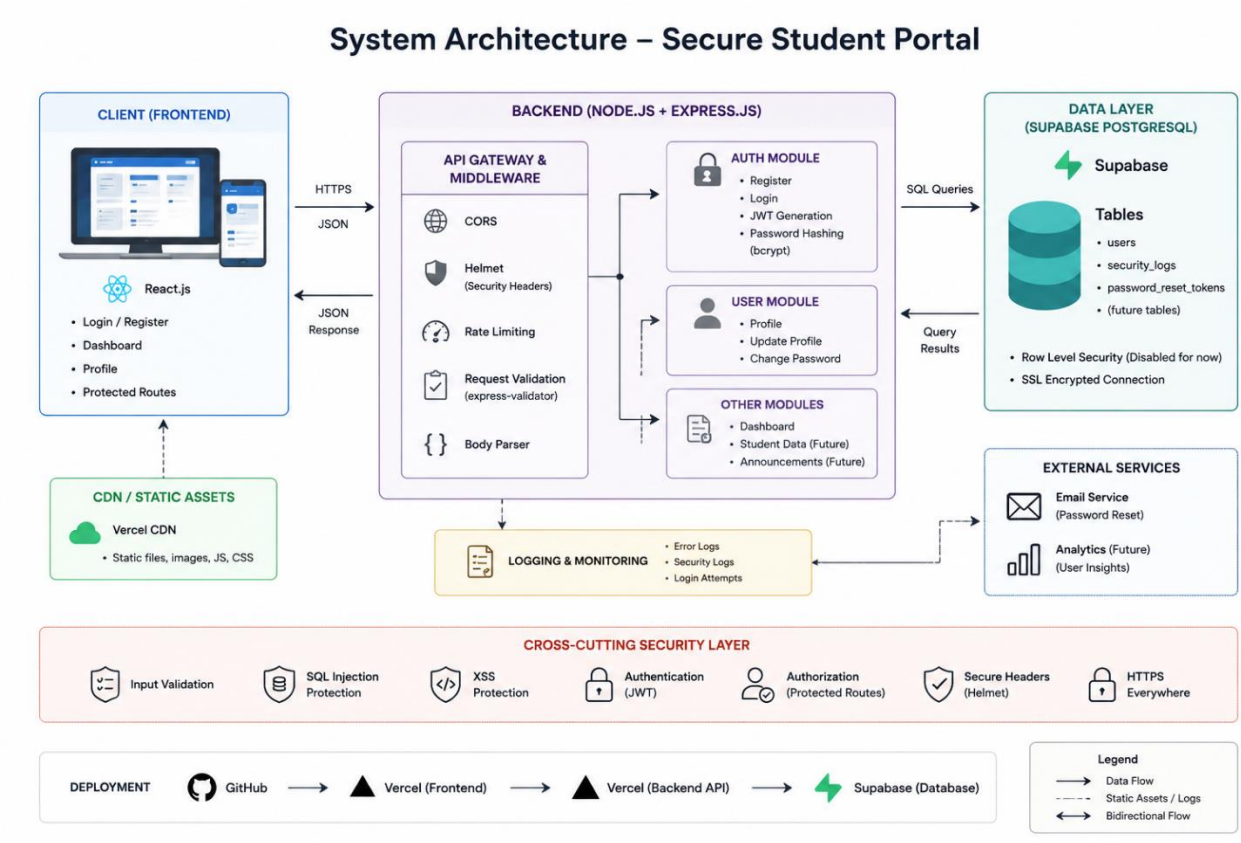
- bcrypt
- jsonwebtoken
- express-rate-limit
- helmet
- validator

Deployment

- Vercel
- GitHub
- Supabase

---

6. SYSTEM ARCHITECTURE



Frontend (React)

↓

Backend API (Express)

↓

Supabase PostgreSQL Database

---

7. DATABASE DESIGN

Users Table

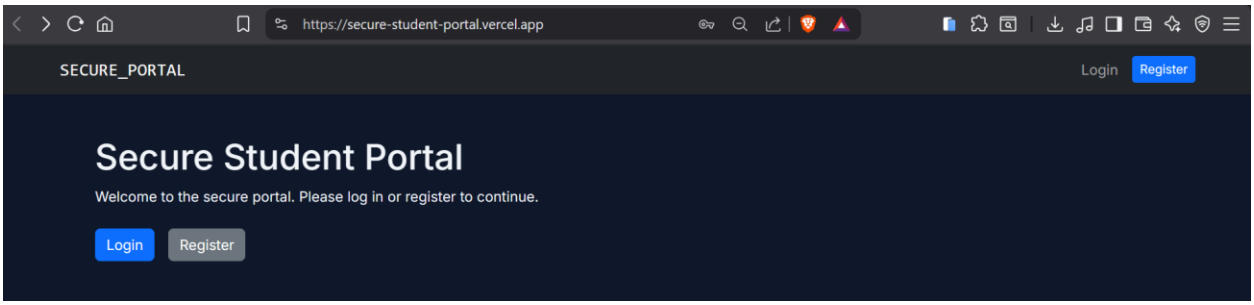
Column	Type
id	UUID
name	VARCHAR
email	VARCHAR
password_hash	TEXT
role	VARCHAR
created_at	TIMESTAMP

Security Logs Table

Column	Type
id	UUID
event_type	VARCHAR
user_email	VARCHAR
ip_address	VARCHAR
timestamp	TIMESTAMP

---

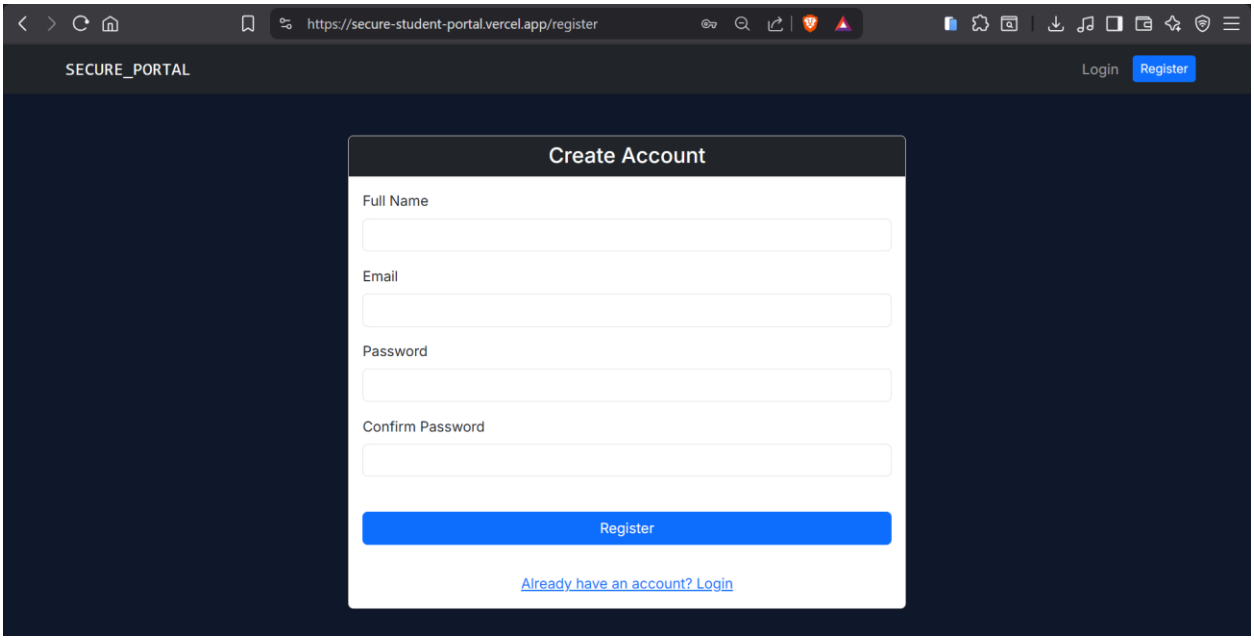
8. IMPLEMENTATION DETAILS



User Registration Module

Features:

- Name validation
- Email validation
- Password strength validation
- Password hashing using bcrypt

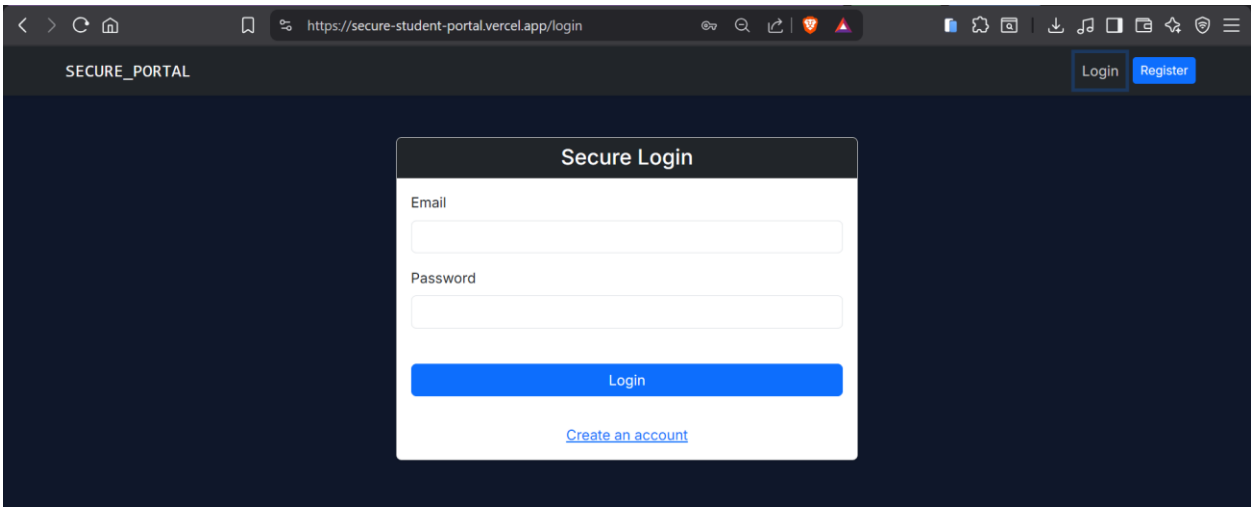


User Login Module

Features:

- Credential verification
- JWT token generation

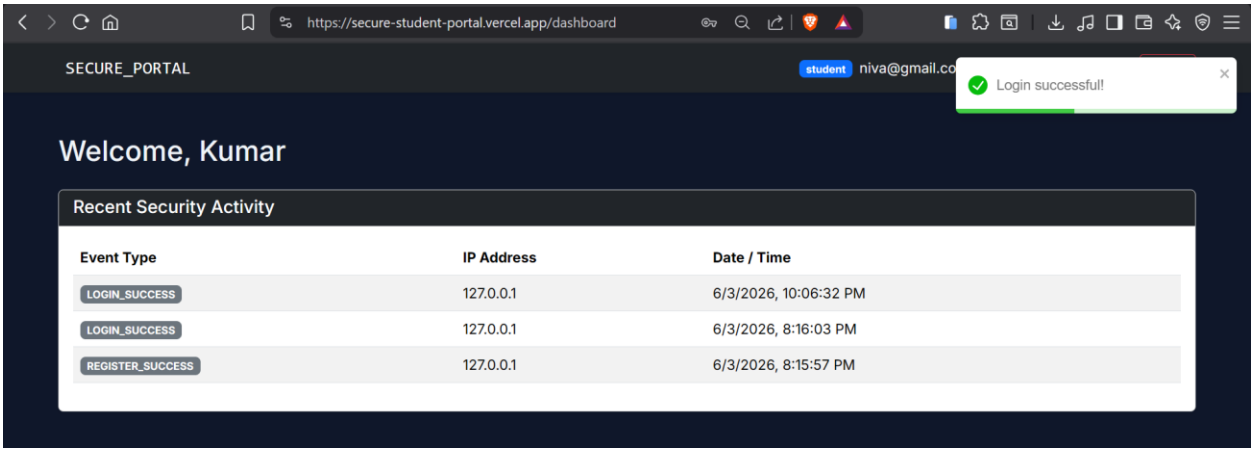
- Secure authentication



User Dashboard

Features:

- Protected routes
- Session validation
- User profile access

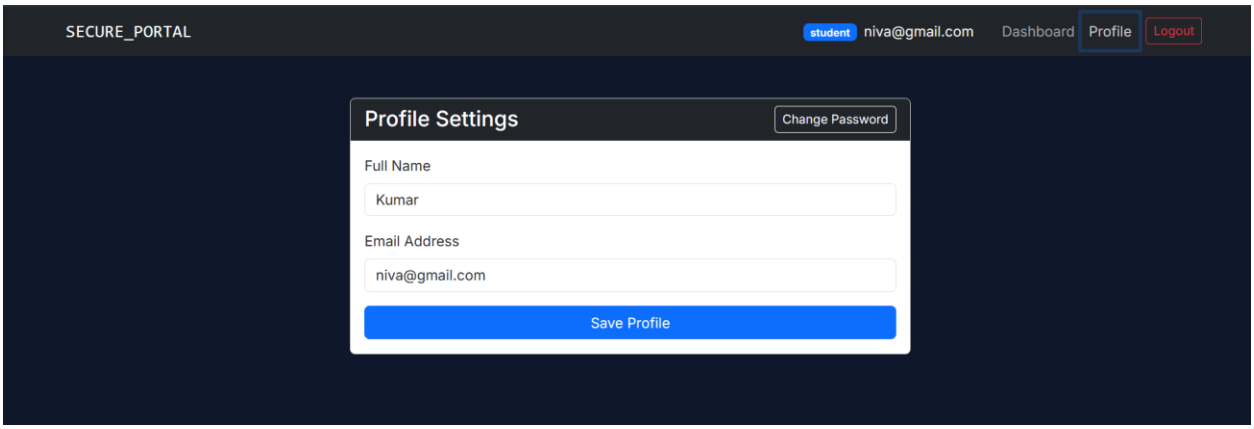


Profile Management

Features:

- Update profile details
- Secure data validation

- Authorization checks

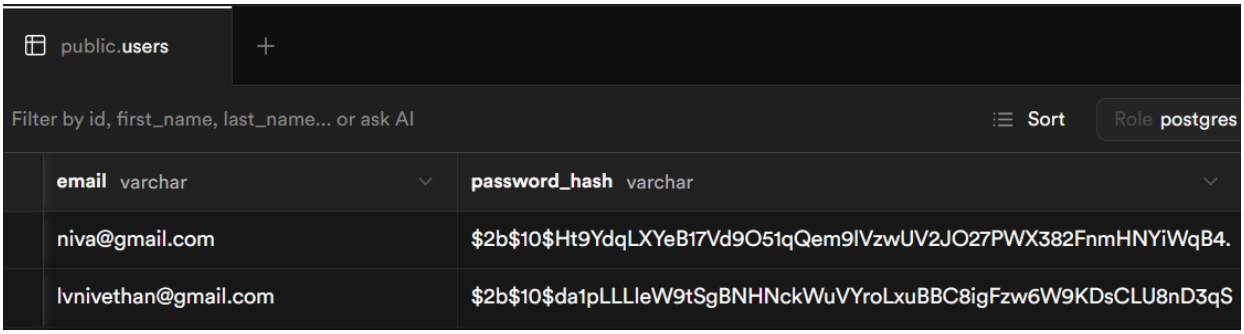


---

9. SECURITY FEATURES IMPLEMENTED

Password Hashing

Passwords are hashed using bcrypt before being stored in the database. Plain-text passwords are never stored.

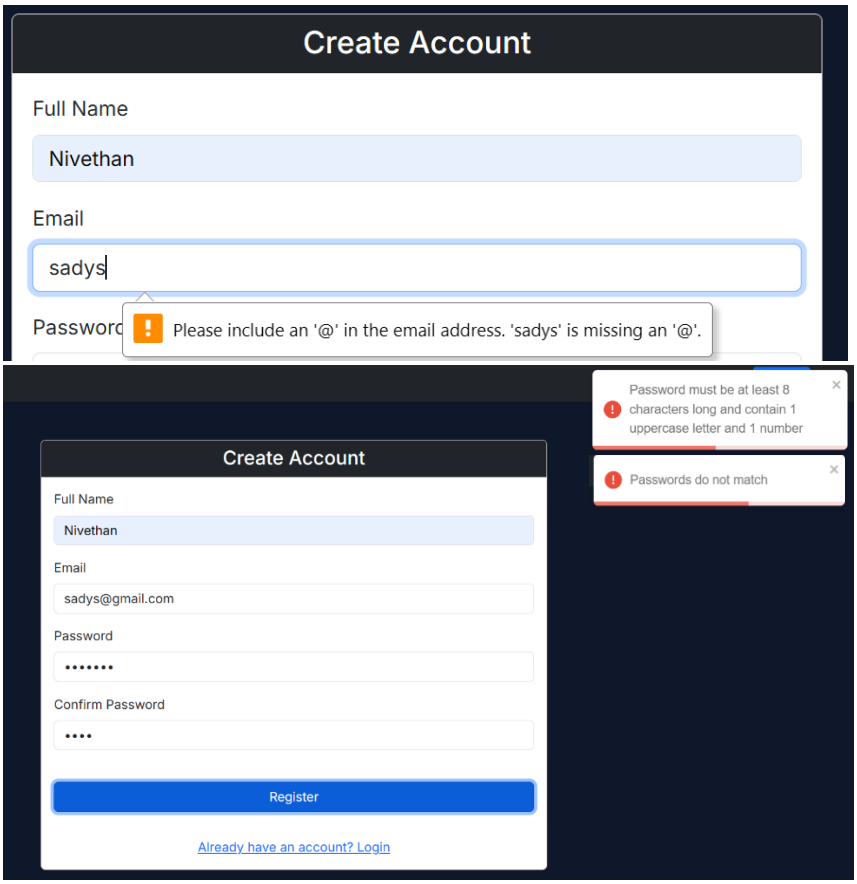


Input Validation

Both client-side and server-side validation are implemented.

Validated Fields:

- Name
- Email
- Password



SQL Injection Prevention

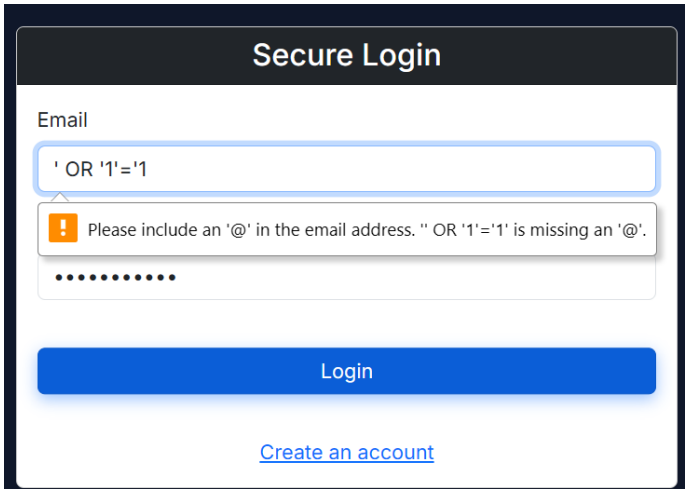
Parameterized database queries are used throughout the application.

Example Attack Tested:

’ OR ‘1’=’1

Result:

Attack Successfully Blocked.



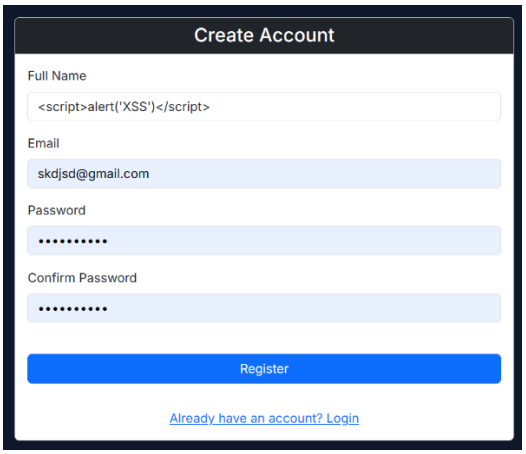
Cross-Site Scripting (XSS) Prevention

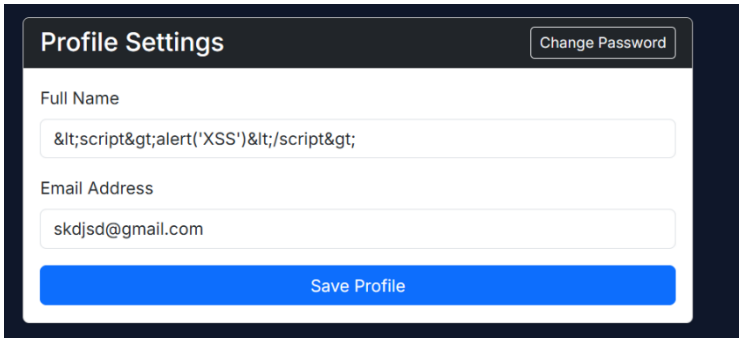
User input is sanitized before processing.

Example Payload:

Result:

Script Execution Prevented.



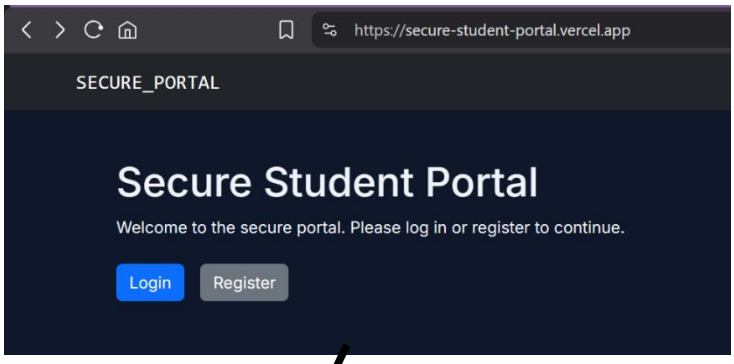


JWT Authentication

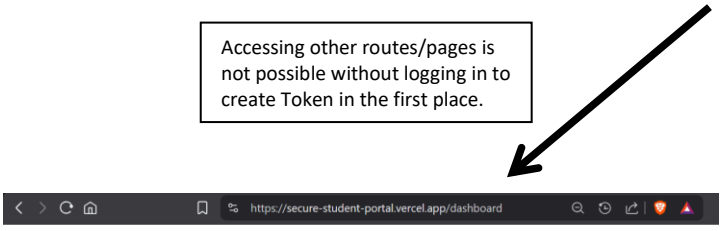
Secure JWT-based authentication is used for protected routes.

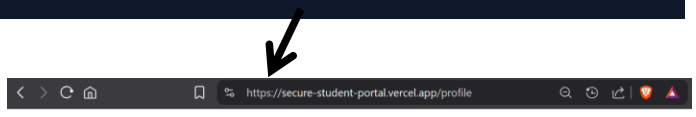
Features:

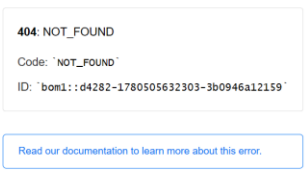
- Token Verification
- Token Expiration
- Unauthorized Access Prevention

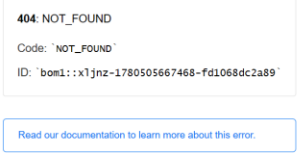


Accessing other routes/pages is not possible without logging in to create Token in the first place.







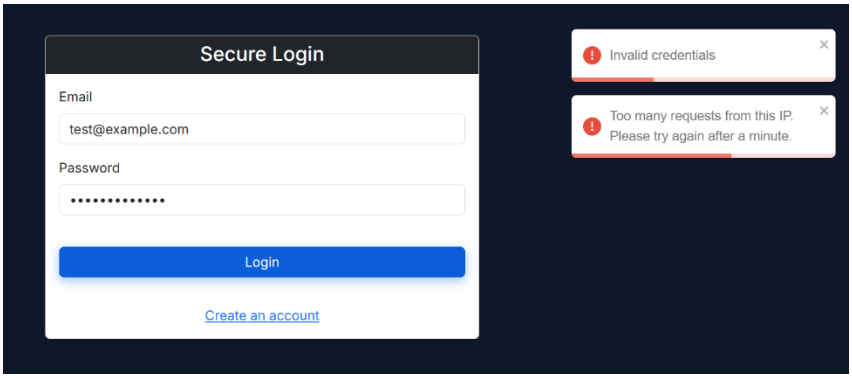


Rate Limiting

Login endpoints are protected against brute-force attacks.

Features:

- Request Limiting
- Temporary Access Restriction

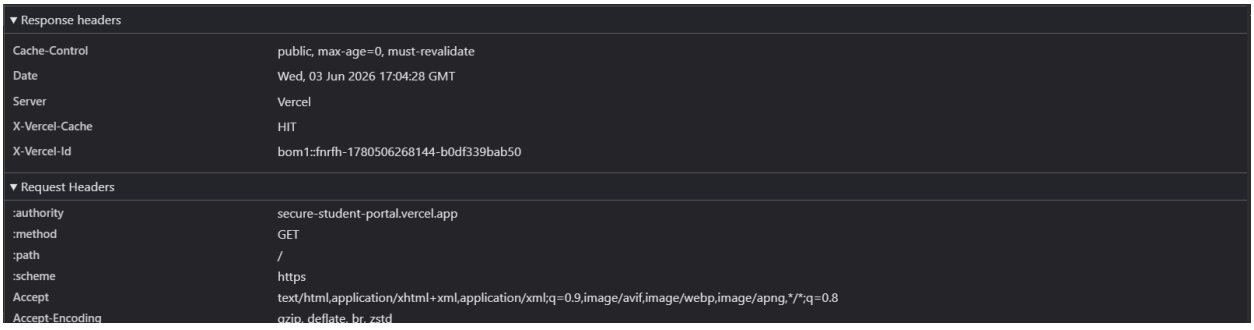


Security Headers

Implemented using Helmet middleware.

Headers Include:

- Content Security Policy
- X-Frame-Options
- X-Content-Type-Options



---

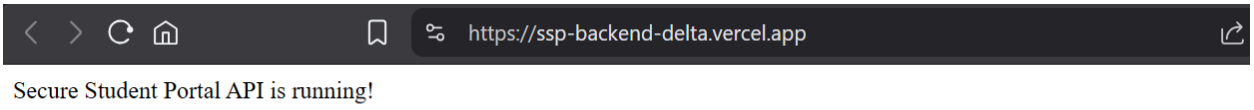
10. OWASP TOP 10 MAPPING

OWASP Risk	Mitigation Implemented
A01 Broken Access Control	Protected Routes & JWT
A02 Cryptographic Failures	bcrypt Password Hashing
A03 Injection	Parameterized Queries
A04 Insecure Design	Validation & Secure Architecture
A05 Security Misconfiguration	Helmet & Environment Variables
A06 Vulnerable Components	Updated Dependencies
A07 Authentication Failures	Strong Password Policies
A08 Software Integrity Failures	Controlled Dependencies
A09 Logging Failures	Security Logging
A10 SSRF	No External URL Processing

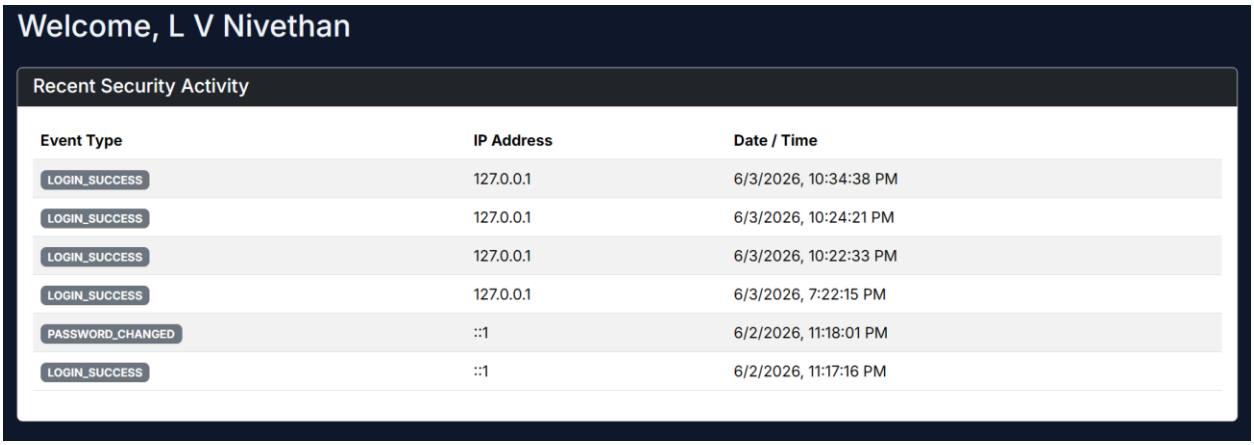
---

11. MORE SCREENSHOTS

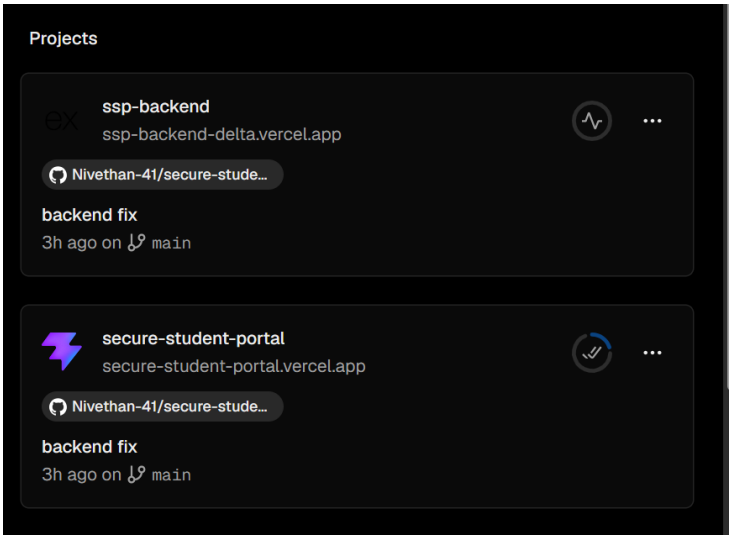
Backend API Verification



Security Logs



Vercel Deployment



---

12. TESTING

Functional Testing

Test Case	Expected Result	Status
Registration	User Created	Pass
Login	Authentication Success	Pass
Logout	Session Ends	Pass
Profile Update	Data Updated	Pass

Security Testing

Security Test	Result
SQL Injection	Prevented
XSS	Prevented
Unauthorized Access	Prevented
Invalid JWT	Rejected
Brute Force Attempts	Rate Limited

---

13. CHALLENGES FACED

- Configuring secure authentication.
- Managing JWT sessions.
- Deploying frontend and backend separately.
- Connecting Supabase with Express.
- Handling CORS configuration.
- Testing security controls effectively.

---

14. FUTURE ENHANCEMENTS

- Multi-Factor Authentication (MFA)
- Password Reset via Email
- Account Recovery Mechanism
- Admin Dashboard
- Advanced Audit Logging
- Security Monitoring Dashboard
- Role-Based Access Control

---

15. CONCLUSION

The Secure Student Portal project successfully demonstrates secure web application development practices. The application implements authentication, authorization, password hashing, session management, input validation, and protection against common web vulnerabilities.

The project highlights the importance of integrating security throughout the development lifecycle and provides practical experience in implementing cybersecurity concepts in a real-world web application.

---

16. REFERENCES

1. OWASP Top 10 Documentation
2. React Documentation
3. Node.js Documentation
4. Express.js Documentation
5. Supabase Documentation
6. Vercel Documentation
7. PostgreSQL Documentation
8. bcrypt Documentation
9. JWT Documentation
10. Helmet Documentation